AI · DATA ANALYTICS · FULL-STACK

Turn a spreadsheet into a conversation.

A portfolio project that lets people upload tabular data and explore it by asking questions in everyday language.

YOUR 45-SECOND INTRODUCTION

> "I built **Cloud Analyst**, a full-stack AI data analysis app. A user uploads a CSV or Excel file, the backend loads it into a Pandas DataFrame, and then the user can ask questions in plain English. A LangChain dataframe agent uses a Groq-hosted language model and Python tools to compute answers from the data. The React interface presents the results in a chat experience with readable Markdown tables and lists. I focused on making data exploration accessible without requiring users to write SQL or Python."

PROJECT AT A GLANCE

Problem

Non-technical users often need help extracting insights from spreadsheets.

Solution

Upload a dataset and ask natural-language questions; get computed, structured responses.

Audience

Analysts, students, and business users exploring small-to-medium tabular files.

TECHNOLOGY STACK

React 19 · Vite · Flask · Pandas · LangChain · Groq API · Qwen model · React Markdown

WHAT IT DOES TODAY

Dataset upload

Accepts CSV and Excel formats, reads the file into Pandas, and returns row count, column count, names, and a five-row preview.

Conversational analysis

Routes a question to a dataframe agent, executes calculations against the active DataFrame, and returns Markdown-formatted findings.

HOW THE APPLICATION WORKS

Architecture & request flow

The browser handles the user experience; Flask handles file parsing and analysis; the model helps interpret questions and select dataframe operations.

1 · React UI
Choose a file
Ask a question

→

2 · Flask API
Validate request
Parse CSV / Excel

→

3 · Pandas
Represent dataset as a DataFrame

→

4 · Agent + LLM
Choose tools
Compute results

→

5 · Chat response
Return JSON
Render Markdown

END-TO-END EXPLANATION

1. **Select and upload:** The user selects a .csv, .xlsx, or .xls file. React sends it as multipart form data to `POST /api/upload`.
2. **Validate and parse:** Flask checks for a file and filename, sanitizes the name with Werkzeug, saves it, and uses Pandas to read the supported format.
3. **Return dataset metadata:** The API sends a JSON summary with the filename, row and column counts, column names, and a small preview. The UI displays the active dataset.
4. **Ask a question:** React sends JSON containing the question to `POST /api/query`. The server creates a dataframe agent for the active DataFrame.
5. **Compute and explain:** The agent uses Python dataframe operations to calculate an answer. The LLM writes a concise response; Markdown is rendered in the chat.

MAIN API ENDPOINTS

`GET /`	Lightweight health check; returns a JSON status message.
`POST /api/upload`	Multipart upload; responds with dataset summary or a validation/parsing error.
`POST /api/query`	Accepts `{"query":"..."}`; responds with the original question and the agent result.

WHY USE AN AGENT?

A regular chat model may guess at numeric answers. The dataframe agent can call a Python tool that works against the actual DataFrame, then describe the result. The app's prompt instructs it to avoid invented values and to use concise Markdown, key findings, and comparison tables when appropriate.

TECHNICAL DECISIONS

What to highlight in the interview

Why React + Vite?

React keeps upload, chat, loading, error, and dataset-summary UI state in focused components. Vite provides a quick local development server and production build.

Why Flask?

The backend is a small JSON API that is straightforward to connect to the frontend. Flask-CORS enables local frontend/backend development across ports.

Why Pandas?

Pandas provides familiar tabular operations and supports both CSV and Excel parsing. The DataFrame is the working representation for analysis.

Why Markdown?

Markdown supports readable headings, emphasized results, bullet lists, and tables. React Markdown plus GFM turns model text into formatted chat content.

RELIABILITY AND USER EXPERIENCE

- Checks for missing files, empty filenames, unsupported formats, missing datasets, and blank questions.
- Uses `secure_filename` to sanitize uploaded filenames.
- Shows upload/query loading states and surfaces API/network errors in the interface.
- Displays a dataset preview and row/column metadata before users ask questions.
- Uses a bounded model output length and a model available to the configured Groq account.

HOW TO DESCRIBE YOUR CONTRIBUTION

> "I connected a React chat interface to a Flask API, implemented file upload and tabular parsing with Pandas, integrated a LangChain dataframe agent with a hosted LLM, and improved response readability with structured Markdown. I also traced integration errors across the API, model, and UI and validated the complete flow with a sample CSV."

DEMO PLAN (ABOUT 2 MINUTES)

1. Introduce the problem: spreadsheet insights should not require writing code.
2. Upload a small sample CSV and point out its row count, columns, and preview.
3. Ask a concrete question tied to those columns, such as "Which category has the highest total?"
4. Show the direct answer, comparison table, and key findings.
5. Close with current limitations and one production improvement you would make next.

Tip: Bring a small, clean sample CSV with obvious results. Confirm the API key is configured privately and both local servers start before the interview.

PRACTICE YOUR ANSWERS

Likely questions & honest answers

What problem does the project solve?

It makes basic exploration of tabular files more accessible. Instead of writing a script for every question, a user can upload a file and ask for calculations or patterns in plain language.

How do you make answers data-grounded?

The dataframe agent has a Python tool connected to the active Pandas DataFrame. It can calculate from the dataset rather than relying only on the language model's memory. I still treat model output as something to verify, especially for consequential decisions.

What happens if the user uploads an invalid file?

The server checks the request and extension, then Pandas attempts to parse the file. The API returns an error response, and the frontend displays the message. A production version should validate file size, content, and schema more thoroughly.

How is the API key handled?

The key is loaded from an environment variable using dotenv and is not included in frontend code. For deployment, I would use a managed secret store, rotate keys, and make sure secrets never enter source control or logs.

What are the current limitations?

The current backend stores the active DataFrame in a process-wide global variable, so it is a single-user/demo design and concurrent users could interfere. Uploads are stored on local disk, there are no user accounts or persistent chat sessions, and arbitrary code execution by the dataframe agent needs stronger isolation before production use.

What would you improve next?

I would add per-user/session dataset storage, size and type limits, authentication and rate limiting, a sandboxed analysis worker, structured logging and tests, deployment configuration, and clearer controls for removing uploaded files. For large datasets, I would avoid loading everything into one in-memory DataFrame.

How did you test it?

I tested the upload and query endpoints with a temporary sample CSV, verified the response status and calculated result, and ran a production build of the frontend. The next step is adding automated unit and integration tests to make those checks repeatable.

A THOUGHTFUL CLOSING STATEMENT

> "This is a working prototype focused on a clear user workflow. I understand the difference between a demo and a production-ready analytics service, especially around isolation, multi-user state, file security, and observability. Those are the first areas I would address as I scale it."

PROJECT FILES TO RECOGNIZE

Frontend: `frontend/my-react-app/src/App.jsx`
Chat, upload, state, Markdown UI

Backend: `backend/uploads/app.py`
Flask routes, parsing, model/agent setup